

CONSIGNES 1

- Vous rédigerez chaque exercice sur une copie séparée.
- Mettez votre nom sur l'énoncé et rendez l'énoncé à la fin du DS.
- Les 3 exercices de ce DS sont indépendants.
- L'énoncé est volontairement long mais il n'est pas nécessaire d'avoir tout fait pour avoir 20!

Exercice 1

On souhaite écrire un algorithme permettant de trier par ordre croissant des tableaux dont les valeurs sont prises parmi un ensemble fini E qui est connu à priori. Nous allons commencer par supposer que E ne contient que 2 valeurs, par exemple $E = \{0, 1\}$. Ainsi, le résultat du tri du tableau $[0, 1, 1, 0, 0]$ sera $[0, 0, 0, 1, 1]$.

Question 1. Écrire un algorithme $\text{TRI2}(tab)$ qui trie efficacement un tableau tab dont les éléments appartiennent à $E = \{0, 1\}$. On pourra par exemple utiliser une méthode de comptage.

Question 2. Quelle est la complexité de votre algorithme ? Justifiez votre réponse.

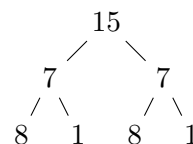
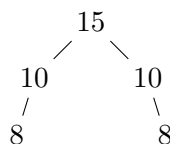
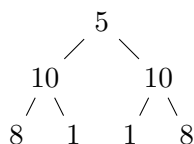
Question 3. Améliorer votre algorithme pour que le tri soit réalisé en **un seul** parcours du tableau.

Question 4. Écrire un algorithme $\text{Tri}(tab, k)$ qui trie la tableau tab dont les éléments appartiennent à $E = \{0, 1, \dots, k-1\}$.

Exercice 2

Les 2 questions de cet exercice sont indépendantes.

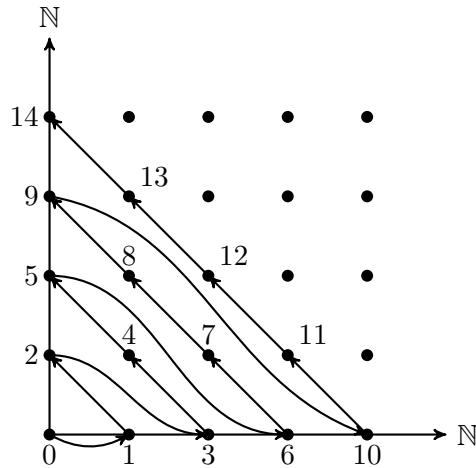
Question 1. Un arbre binaire est dit symétrique s'il est vide ou si ses deux sous-arbres sont miroirs l'un de l'autres. Ainsi, parmi les arbres suivants, les deux premiers sont symétriques mais pas le troisième :



Écrire un algorithme `SYMETRIQUE(a)` qui renvoie `true` si l'arbre a est symétrique.

Indication : Pour écrire cet algorithme, vous pourrez définir et vous servir d'une fonction auxiliaire qui détermine si deux arbres binaires sont miroirs l'un de l'autre.

Question 2. On démontre que l'ensemble des paires d'entiers naturels (dénnoté $\mathbb{N} \times \mathbb{N}$) peut être énuméré (ie. qu'il est *dénombrable*) en numérotant chaque couple $(x, y) \in \mathbb{N} \times \mathbb{N}$ suivant le procédé suggéré par la figure suivante.



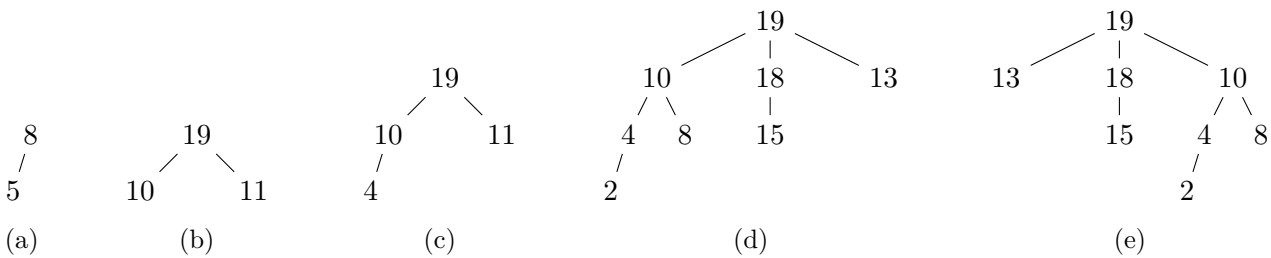
- soit constitué d'une feuille et, dans ce cas, son ordre est $k = 0$,
- soit constitué d'une racine ayant k fils qui sont des arbres binomiaux d'ordre respectivement $k - 1, k - 2, \dots, 1, 0$ (de gauche à droite).

On admettra les propriétés suivantes :

- un arbre binomial d'ordre k a une hauteur k ;
- un arbre binomial d'ordre k possède 2^k noeuds.

Un *tas binomial* est un arbre binomial dont les éléments respectent la contrainte de tas (max) : la valeur d'un noeud est toujours supérieure ou égale à celle de ses fils. Pour implémenter un tas binomial α d'ordre k , on utilisera une structure de données arborescente (de noeuds chaînés) : *NIL* représente le tas binomial vide, et, quand α est un noeud, l'attribut $\alpha.val$ représente la valeur stockée dans α , l'attribut $\alpha.ordre$ représente l'ordre du tas enraciné en α , et $\alpha.fils[i]$ représente le $(i + 1)$ ème fils de α .

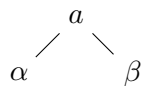
Question 2. Pour chacun des arbres suivants, dire s'il s'agit d'un tas binomial. Si oui, donner son ordre. Si non, justifier.



Nous allons maintenant nous intéresser à la conversion d'un tas binaire en un tas binomial. Pour que cette conversion soit possible, d'après les propriétés précédentes, il est évidemment nécessaire que le nombre de noeuds du tas binaire à convertir soit une puissance de 2. Ainsi, on pourra convertir un tas binaire de taille 2^k en un tas binomial d'ordre k .

Question 3. Dessiner des **tas binaires** dont le nombre de noeuds est 1, 2^2 et 2^3 . En général, que peut-on dire de la forme des tas binaires de taille 2^k ? Si $k \geq 1$, quels sont les nombres de noeuds des sous-arbres gauche et droit d'un tas binaire de taille 2^k ?

Question 4. Soit un tas binaire de taille 2^k :



où α et β sont, respectivement, les sous-arbres gauche et droit, et a est la valeur à la racine du tas. Donner un algorithme *TASDROIT*(a, β) qui insère la valeur a dans le sous-arbre β et renvoie un **tas binaire**. On notera β' le tas ainsi obtenu.

Question 5. Donner la complexité de cet algorithme.

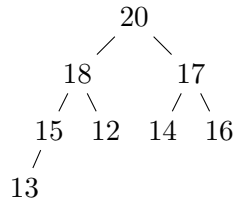
Question 6. Pourquoi l'algorithme *TASDROIT* peut être réalisé sans faire aucune comparaison ?

Question 7. En supposant qu'on ait transformé les **tas binaires** α et β' (tous deux de taille 2^{k-1}) en **tas binomiaux** α' et β'' , donner un algorithme *FUSION* qui les fusionne en un seul tas binomial d'ordre k . Un peu plus généralement, on écrira une fonction *FUSION*(γ_1, γ_2), où γ_1 et γ_2 sont des tas binomiaux de même ordre k , qui renvoie un tas binomial d'ordre $k + 1$.

Indication : La racine du tas renvoyé sera la plus grande des deux racines de γ_1 et γ_2 , il reste à déterminer quels seront les $k + 1$ fils de cette racine...

Question 8. Quel est le coût de l'opération précédente ?

Question 9. Donner un algorithme récursif BINAIRE2BINOMIAL(α) qui, étant donné un tas binaire α de taille 2^k (où $k \geq 0$) le transforme en tas binomial. On supposera qu'on dispose de deux fonctions FILS GAUCHE(α) et FILS DROIT(α) qui, à un **tas binaire** α non vide stocké sous forme de tableau, renvoie le tas binaire fils gauche, respectivement droit, de α sous forme de tableau. Par exemple, le tas binaire



est représenté sous-forme de tableau par : $\alpha =$

20	18	17	15	12	14	16	13
----	----	----	----	----	----	----	----

L'appel FILS GAUCHE(α) renverra le tableau :

18	15	12	13
----	----	----	----

Question 10. Appliquer votre algorithme au tas binaire suivant :

